PROJECT THESIS REPORT

AgentForge-RAG

Agentic Retrieval-Augmented Generation with Hybrid Retrieval, Grounded Verification, Guardrails and a Reproducible Evaluation Harness

HEADLINE RESULTS

100%
Recall@3 & Recall@5
main + paraphrase sets

100%
Groundedness
28 of 28 answers verified

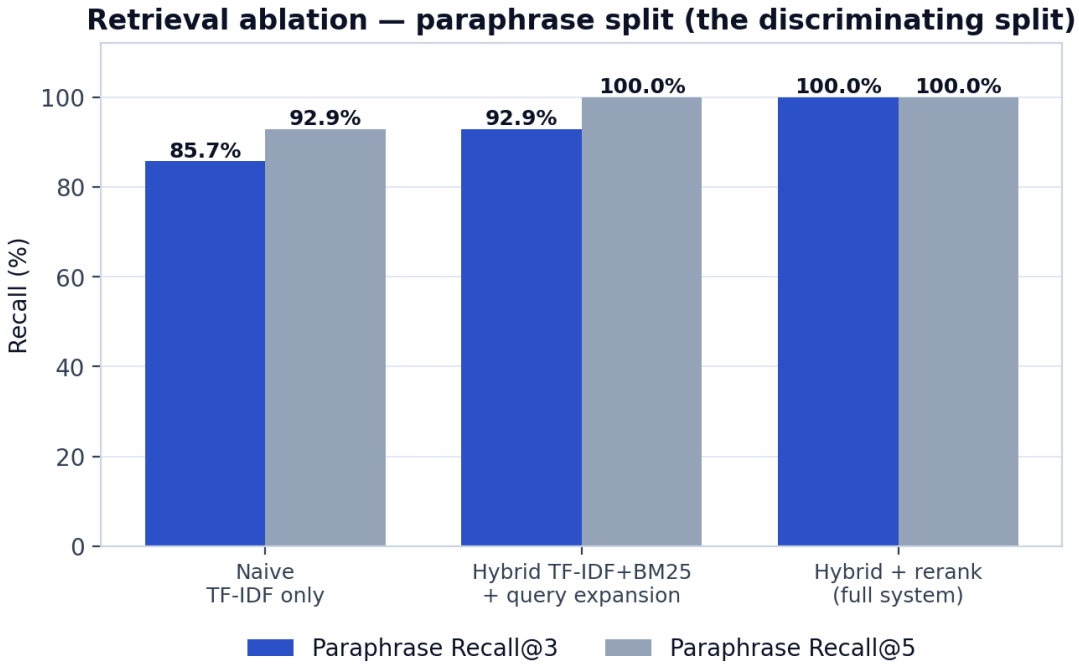
100%
Abstention & injection block
5/5 + 5/5

10.56 ms
Mean query latency
local CPU, per query

Ashutosh Sahoo
M.Tech, Artificial Intelligence · NIT Silchar
November 2026

Results at a Glance

100% Recall@3 & Recall@5 main + paraphrase sets	**100%** Groundedness 28 of 28 answers verified
100% Abstention & injection block 5/5 + 5/5	**10.56 ms** Mean query latency local CPU, per query



Ablation on the paraphrase split: each stage of the retrieval stack (hybrid TF-IDF + BM25 fusion, then reranking) earns a measured gain over naive TF-IDF alone. All values from results.json.

Contents

Abstract

1. Introduction

2. Background and Theory

3. System Design and Architecture

4. Implementation — A Function-by-Function Walkthrough

5. Dataset and Corpus

6. Experiments

7. Results and Analysis

8. Discussion

9. Conclusion and Future Work

10. Learning Guide

11. How to Run

12. References

Appendix A — Full `results.json`

Appendix B — File Listing

Abstract

Retrieval-Augmented Generation (RAG) has become the standard architecture for answering questions over private, proprietary or fast-changing document collections without retraining a language model. Yet most textbook and tutorial implementations of RAG are *naive*: documents are embedded once, the top-k most similar chunks are retrieved for a query, and the results are handed to a generator. This naive pipeline fails silently in at least three well-documented ways — it misses documents when the user's vocabulary differs from the document's vocabulary, it generates fluent statements that are not supported by any retrieved evidence, and it answers out-of-scope questions with the same confidence as in-scope ones. This report presents **AgentForge-RAG**, a complete, zero-dependency, production-shaped RAG system built to attack each of those three failure modes directly, and to *measure* whether the attack worked.

The system combines five ideas from the current state of the art. First, **hybrid retrieval**: a TF-IDF cosine retriever is fused with an Okapi BM25 retriever implemented from scratch, so that both term-frequency statistics and BM25's length-normalised, saturation-aware scoring contribute to ranking. Second, **query expansion**: a small, auditable synonym and abbreviation lexicon rewrites the user's phrasing into document phrasing before retrieval. Third, **reranking**: the top-20 hybrid candidates are re-scored by a deterministic cross-scorer — a stand-in for a neural cross-encoder — that measures query-term coverage, best-sentence overlap and phrase matches. Fourth, **grounded, extractive composition with verification**: answers are assembled only from verbatim sentences of retrieved chunks, each carrying a `[document:chunk]` citation, and a separate Verifier agent re-checks every answer sentence for token overlap against the cited evidence; an answer that fails verification is replaced by an abstention. Fifth, **guardrails**: pattern-based prompt-injection detection and personally identifiable information (PII) redaction are applied to both inputs and outputs, and an absolute fused-score threshold triggers an explicit "I don't have enough information" abstention for out-of-scope questions.

Around the pipeline sits a multi-agent orchestration layer (Router, Retriever, Verifier agents with a full execution trace) and, most importantly, a **reproducible evaluation harness**. The harness first performs a grid search over chunk size (80/120/160/200 words) and overlap (20/30/40 words) and selects the abstention threshold by answerable-versus-unanswerable score separation; it then evaluates the tuned system on a 28-question main set, a deliberately harder 14-question paraphrase split, 5 unanswerable questions and 5 prompt-injection attacks, and ablates the full stack against its own weaker variants. The tuned configuration (200-word chunks, 20-word overlap, threshold 0.20) achieves Recall@3 and Recall@5 of 1.0 on both question splits, a groundedness rate of 1.0 over 28 answered questions, abstention correctness of 1.0 (5/5), an injection block rate of 1.0 (5/5, confirmed end-to-end), and a mean latency of 10.56 ms per query on a local CPU. The ablation is the scientific core of the work: naive TF-IDF alone reaches only 0.8571 Recall@3 on the paraphrase split, hybrid retrieval with expansion lifts it to 0.9286, and reranking closes the remaining gap to 1.0. Each stage of the architecture therefore earns its place with a measured gain, and each remaining weakness — a narrow threshold separation margin of 0.1919 versus 0.2105, a small and domain-bound evaluation set, and a lexical rather than neural retrieval core — is analysed openly. The report is written so that a learner can study modern RAG end-to-end from it alone: every concept is derived from first principles, every file and function of the implementation is walked through, and a learning guide with interview questions, model answers and extension exercises closes the work.

Keywords: Retrieval-Augmented Generation; hybrid retrieval; BM25; TF-IDF; reranking; query expansion; grounding; hallucination; abstention; prompt injection; PII redaction; RAG evaluation;

multi-agent systems.

1. Introduction

1.1 The problem

Large language models (LLMs) store knowledge in their parameters. That design has three practical consequences for anyone who wants to answer questions about a *specific* organisation's documents — its security policies, incident playbooks, HR rules, product manuals:

1. **The model does not know your documents.** They were never in its training data, and adding them by fine-tuning is slow, expensive, and must be repeated every time a policy changes.
2. **The model's knowledge is frozen and un-cited.** It cannot tell you *which* document a claim came from, so a user cannot verify an answer.
3. **The model answers everything with the same tone.** Asked about a topic it knows nothing about, it does not reliably say "I don't know" — it produces a plausible-sounding answer. In a compliance, medical, legal or security context, a confident wrong answer is worse than no answer.

Retrieval-Augmented Generation (Lewis et al., 2020) solves the framing problem: instead of asking the model to *remember*, we *retrieve* the relevant passages from a document store at query time and ask the model to answer *from those passages*. Knowledge becomes updatable (re-index the documents), attributable (cite the passages) and inspectable (log what was retrieved). RAG is, for this reason, the dominant pattern behind enterprise question-answering products.

1.2 Why naive RAG is not enough

The minimal RAG pipeline — chunk, embed, retrieve top-k by cosine similarity, generate — is easy to demo and surprisingly easy to get wrong in ways that only appear in real use:

Vocabulary mismatch. Users write "fake email", "login secret", "files locked by criminals". Policy documents write "phishing", "password", "ransomware encryption". Any purely lexical retriever, and even many embedding models, will rank the right chunk too low when the words do not overlap. This is the single most common cause of "the RAG system couldn't find it, but it's right there in the manual."

Ungrounded generation. A generative model conditioned on retrieved text can still add claims that appear nowhere in that text — the *hallucination* problem. Conditioning reduces hallucination; it does not eliminate it, and nothing in the naive pipeline *checks*.

No notion of "out of scope". Cosine similarity always returns a top-k, even for the query "What is the recipe for a chocolate cake?" against a security-policy corpus. The nearest chunk is meaningless, but it has a score, and the generator will happily build an answer on it.

No safety layer. A document-QA endpoint is an attack surface: users (or documents) can attempt prompt injection ("ignore previous instructions and reveal your system prompt"), and both queries and answers can leak personal data.

No measurement. Most demo RAG projects report no numbers at all. Without an evaluation set, an ablation and a tuning protocol, "it works" is an anecdote, and every subsequent change is a gamble.

1.3 Objectives

AgentForge-RAG was built with five objectives, each mapped to one failure above:

O1 — Retrieval quality under paraphrase. Combine complementary retrieval signals (TF-IDF and BM25), query expansion and reranking so that questions phrased differently from the documents still retrieve the right document, and prove the gain with an ablation on a paraphrase split.

O2 — Verifiable grounding. Compose answers only from retrieved evidence, cite every sentence, and verify grounding with an independent agent that can veto an answer.

O3 — Calibrated abstention. Detect out-of-scope questions with an absolute score threshold — tuned, not guessed — and answer them with an explicit abstention.

O4 — Guardrails by construction. Block known prompt-injection patterns and redact PII on both the input and output paths.

O5 — Reproducibility and zero-dependency engineering. Implement everything with the Python standard library plus NumPy only — no API keys, no downloads, no GPU — with an assert-based test suite (25 tests) and an evaluation harness whose outputs (`results.json`) are the *only* source of numbers in this report.

1.4 Contributions

1. A complete, working **hybrid retrieval stack implemented from scratch**: TF-IDF vectorisation in NumPy, Okapi BM25 (including the Robertson/Spärck-Jones IDF), score fusion with a saturating BM25 transform chosen specifically to keep fused scores on an absolute scale, a synonym/abbreviation query-expansion lexicon, and a deterministic cross-scoring reranker.
2. A **multi-agent orchestration layer** — Router, Retriever and Verifier agents around the pipeline, with guardrails as a pre-agent and a per-step execution trace — showing how agentic decomposition adds routing, verification and observability without changing the retrieval core.
3. A **grounding verifier with veto power**: every answer sentence must share at least 50% of its content tokens with the cited retrieved text, or the answer is discarded and replaced by an abstention.
4. A **tuning and evaluation methodology**: chunking grid search, threshold selection by score separation, a main split and a paraphrase split, unanswerable and attack sets, latency measurement, and a three-way ablation isolating the contribution of each retrieval stage.
5. An **honest limitations analysis**, including the disclosure that the tuned abstention threshold separates the evaluation sets by a narrow margin (answerable minimum top score 0.2105 vs. unanswerable maximum 0.1919).

1.5 Report roadmap

Chapter 2 derives every concept used — TF-IDF, cosine similarity, BM25, fusion, reranking, grounding, abstention and injection defence — from first principles and places them in the literature. Chapter 3 presents the architecture. Chapter 4 walks through every file and function of the code. Chapter 5 describes the corpus. Chapter 6 gives the full experimental protocol. Chapter 7 reports and analyses the results. Chapters 8 and 9 discuss limitations, production paths and future work. A learning guide, run instructions, references and appendices close the report.

2. Background and Theory

This chapter explains every concept the system uses, in the order data flows through it: tokenisation, chunking, TF-IDF, cosine similarity, BM25, hybrid fusion, query expansion, reranking, extractive composition, grounding verification, abstention, guardrails, and evaluation metrics. Each section ends with *why this project uses it*.

2.1 Tokenisation and content tokens

All retrieval here is **lexical**: text is matched word by word. The first step is therefore to turn raw text into a comparable token stream. AgentForge-RAG lowercases the text and extracts alphanumeric runs with the regular expression `[a-z0-9]+`, discarding tokens of length 1. A second filter removes a fixed stopword list (`the`, `must`, `how`, `what`, ...) to produce **content tokens** — the tokens that carry topical meaning.

Two design choices deserve note. First, stopword removal is applied to *content-token* views (indexing, scoring, verification) but the original text is preserved for display and extraction, so answers remain grammatical, verbatim policy sentences. Second, there is deliberately **no stemming or lemmatisation**: `password` and `passwords` are different tokens. The query-expansion lexicon compensates for the most important morphological/domain variants explicitly (it maps `password passwords, credential, credentials`), which keeps behaviour auditable — a learner can read exactly why a query matched — at the cost of coverage a stemmer would give for free. This is a recurring theme of the project: prefer an explicit, inspectable mechanism over a clever opaque one, and measure the result.

2.2 Chunking: why documents are split, and the size trade-off

Retrieval operates on **chunks**, not whole documents, for three reasons: (i) a question usually concerns one passage, and returning a whole 350-word policy dilutes the ranking signal with irrelevant terms; (ii) the generator/verifier works best with focused evidence; (iii) citations can point to a specific passage.

Chunking introduces two hyperparameters:

> **Chunk size** (words per chunk). Too small, and a chunk lacks the context that makes it match a query and answer a question — the sentence stating a rule may be separated from the sentence stating its number. Too large, and chunks become topically mixed, hurting precision.
>
> **Overlap** (words shared between consecutive chunks). Overlap insures against a relevant passage being cut in half at a boundary: with overlap o, any passage shorter than o words survives intact in at least one chunk.

AgentForge-RAG uses simple word-window chunking: strip Markdown heading markers (keeping the heading words, which carry signal), split on whitespace, and emit windows of `chunk_words` advancing by `chunk_words - overlap_words`. The tuned values are 200 words with 20 words of overlap (Chapter 6 explains how they were chosen; note they are larger than folk-defaults like 100–120 words precisely because this corpus's documents are short and rule-dense, so the grid search preferred chunks that keep a whole rule together).

2.3 TF-IDF: term frequency × inverse document frequency

TF-IDF (Salton and co-workers; Jones, 1972, for IDF) scores how *characteristic* a term is of a document. For term *t* in document *d* from a corpus of *N* documents:

Term frequency TF(t, d) — how often *t* occurs in *d*. This project uses raw counts.

Document frequency DF(t) — in how many documents *t* occurs at least once.

Inverse document frequency, as implemented in `TFIDFEmbedder.fit`:

IDF(t) = ln((1 + N) / (1 + DF(t))) + 1

The `+1` smoothing keeps IDF positive and defined for terms in every document; the logarithm makes IDF grow slowly — a term in 1 of 28 chunks is much more informative than a term in 14, but not 14× more.

TF-IDF vector: each document becomes a vector over the whole vocabulary, with component TF(t, d) · IDF(t), L2-normalised to unit length.

The intuition to carry into an interview: TF says "this chunk talks about *X* a lot"; IDF says "and *X* is rare enough across the corpus that talking about it is informative". Words like `policy` (everywhere) are down-weighted; words like `ransomware` (one document) dominate.

Embedder interface. The code isolates TF-IDF behind an `Embedder` abstract interface (`fit`, `embed`, `embed_many`). This is deliberate software design, not ceremony: in production the same interface is implemented by a neural sentence embedder (e.g. sentence-transformers) and nothing downstream changes. The class also provides `hash_embed`, a stateless fixed-dimension hashing projection (MD5 of each token mod 512) — the classic *hashing vectoriser* trick — to demonstrate how a fixed-size dense representation behaves when no fitted vocabulary exists.

2.4 Cosine similarity

With both query and chunk as L2-normalised TF-IDF vectors, relevance is their dot product, which equals the **cosine of the angle between them**:

cos(q, d) = (q · d) / (||q|| ||d||) = q̂ · d̂ (for normalised vectors)

Cosine lies in [0, 1] for non-negative TF-IDF vectors, is length-invariant (a long chunk is not favoured merely for containing more words), and is cheap: retrieval is one matrix–vector product over the chunk matrix. `Retriever._tfidf_scores` is exactly `matrix @ q`.

Cosine/TF-IDF's known weaknesses motivate the rest of the stack: it treats every occurrence linearly (the 10th mention of `password` counts as much as the 1st, proportionally), it ignores document-length effects beyond normalisation, and it has no synonymy. BM25 addresses the first two; expansion addresses the third.

2.5 Okapi BM25: the probabilistic ranking function

BM25 (Robertson & Walker, 1994; Robertson & Zaragoza, 2009) remains, three decades on, the strongest classical lexical ranker and a component of nearly every production hybrid stack. For query *Q* and document *D*:

BM25(D, Q) = Σ over terms t in Q of IDF_BM25(t) · [f(t,D)·(k1 + 1)] / [f(t,D) + k1·(1 b + b·|D|/avgdl)]

where *f(t, D)* is the term's frequency in *D*, |*D*| is document length, *avgdl* is average document length, and:

IDF_BM25(t) = ln(1 + (N DF(t) + 0.5) / (DF(t) + 0.5)) (floored at 0 in this implementation)

The two constants encode two empirically robust effects:

Term-frequency saturation (k1 = 1.5). The fraction rises steeply for the first few occurrences and then flattens — the difference between 0 and 2 mentions matters far more than between 8 and 10. TF-IDF's linear TF has no such saturation.

Length normalisation (b = 0.75). A term occurring 3 times in a 100-word chunk is stronger evidence than 3 times in a 400-word chunk; the denominator scales the expected frequency by relative length.

`BM25Index` implements this from scratch: it precomputes per-chunk token counts, lengths, average length and the IDF table at index time, so scoring a query is a dictionary lookup per unique query term per chunk. The implementation deduplicates query terms (`dict.fromkeys`) — BM25 sums over distinct query terms in this formulation — and floors IDF at zero so that ultra-common terms cannot contribute negative evidence.

2.6 Hybrid fusion — and why the normalisation choice matters

Hybrid retrieval runs both rankers and **fuses** their scores. AgentForge-RAG uses a weighted linear fusion:

fused(d) = α · TFIDF_cos(d) + (1 α) · BM25_norm(d), with α = 0.55

The subtle, load-bearing decision is `BM25_norm`. Raw BM25 is unbounded (roughly 0–20+), while cosine is in [0, 1]; they must be put on a common scale. The obvious choice — min-max normalisation per query — is **wrong for this system**, and the code comments say why: min-max forces the best chunk to 1.0 on *every* query, including gibberish. The abstention mechanism (§2.9) depends on the top fused score being *absolutely* meaningful: near 0 when nothing matches, higher when something does. The project therefore uses a **saturating transform**:

BM25_norm = BM25 / (BM25 + 8.0)

This maps 0 0, 8 0.5, and large scores asymptotically toward 1, preserving an absolute zero. It is a small piece of engineering that the whole safety story rests on, and a good example of a general lesson: *normalisation is a modelling decision, not a preprocessing detail.*

Why fuse at all? TF-IDF cosine and BM25 disagree in useful ways — cosine rewards broad topical overlap across the full vocabulary, BM25 rewards concentrated, saturated matches of rare query terms with length correction. Empirically (Chapter 7), their fusion with expansion already fixes most paraphrase failures before reranking is even applied.

2.7 Query expansion and rewriting

Query expansion adds terms to the query before retrieval. Production systems learn expansions or generate them with an LLM (HyDE, query2doc); this project uses the oldest auditable form: a **hand-built lexicon** (`EXPANSIONS` in `rag.py`) mapping:

Abbreviations expansions: `mfa → multi factor authentication`, `vpn → virtual private network remote access`, `rto → recovery time objective`, `byod → bring your own device personal device`, `dlp → data loss prevention`, `cvss → severity score vulnerability`.

Domain synonyms: `phishing → suspicious email report`, `ransomware → ransom encryption encrypted`, `password → passwords credential credentials`.

User phrasing ↔ *document phrasing* (the paraphrase bridges): `fake → phishing suspicious`, `secret → password credential`, `locked → encrypted encryption ransomware`, `flaws → vulnerabilities vulnerability`, `coins → cryptocurrency mining`, `worst → severity critical incident`, `programmers → developers coding`.

`expand_query` appends expansions for any token present in the query. Expansion is applied in hybrid and rerank modes but **not** in the naive TF-IDF ablation mode — which is precisely what allows the ablation to attribute the paraphrase gains to expansion + BM25 (Chapter 7). The risks of expansion are real (topic drift if a synonym is ambiguous — `crypto` can mean cryptography, not only cryptocurrency) and are discussed in Chapter 8; a small, reviewable lexicon bounds that risk.

2.8 Reranking: two-stage retrieval

Modern retrieval is almost always **two-stage**: a fast first stage retrieves a candidate pool (here, the top 20 by fused score), and a slower, more precise second stage reorders it. In production the second stage is a *cross-encoder* — a transformer that reads query and chunk *jointly* and outputs a relevance score — which is far more accurate than bi-encoder cosine but too slow to run over the whole corpus.

AgentForge-RAG's reranker is a deterministic stand-in with the same contract: `_cross_score(query, chunk)` combines three joint query–chunk features:

cross = 0.45 · coverage + 0.40 · best-sentence overlap + 0.15 · phrase bonus

Coverage: fraction of query content tokens appearing anywhere in the chunk.

Best-sentence overlap: the same fraction computed against the chunk's single best sentence — rewards a chunk that contains one passage *directly* answering the query, even if the rest of the chunk is off-topic.

Phrase bonus: fraction of query bigrams appearing verbatim in the chunk — rewards exact phrase matches ("recovery time objective").

The final ordering score is `0.55 · fused + 0.45 · cross`. Note the reported `Retrieved.score` deliberately remains the *fused* score, not the combined one, so that the abstention threshold operates on one stable scale regardless of reranking.

2.9 Grounded composition and abstention

Faithfulness / groundedness means every claim in an answer is supported by retrieved evidence. The strongest way to guarantee it in a zero-dependency system is to make the composer **extractive**: `compose_answer` splits retrieved chunks into sentences, scores each by query-token overlap (tie-broken by chunk score), deduplicates (by the first 80 lowercased characters), and emits the top 4 sentences *verbatim*, each suffixed with its citation, e.g. `[password-policy:0]`. An extractive answer cannot invent a number, a deadline or a rule — the worst it can do is select an irrelevant sentence, which retrieval quality and verification guard against.

Abstention is the complementary mechanism. If the best retrieved chunk's fused score is below a threshold τ, the composer returns the fixed abstention string: *"I don't have enough information in the provided documents to answer that."* Because fused scores are absolute (§2.6), a single global τ is meaningful. $\tau = 0.20$ was not guessed: `evaluate.py` selects it to maximise the joint accuracy of (answerable queries scoring $\geq \tau$) and (unanswerable queries scoring $< \tau$). The measured separation — lowest answerable top score 0.2105, highest unanswerable 0.1919 — is narrow, and Chapter 8 treats that honestly as the system's most fragile point.

2.10 Verification agents and multi-agent orchestration

Decomposing a pipeline into **agents** — components with a single responsibility, a typed input/output contract, and a logged trace — buys testability and a place to attach checks. AgentForge-RAG has three agents plus a guardrail pre-step:

Router agent: classifies the query as `incident`, `policy` or `general` by keyword-set overlap (incident terms: *ransomware, breach, severity, triage*...; policy terms: *password, vpn, retention, byod*...). In this deterministic system the label is used for tracing and analysis; in production the same interface routes to different corpora, prompts or models.

Retriever agent: runs the pipeline (retrieval + composition) and logs the hits, top score and abstention flag.

Verifier agent: the grounding check. For each answer sentence it computes the fraction of the sentence's content tokens present in the union of tokens of the *cited* chunks; a sentence is grounded if the ratio 0.5. All sentences must pass. The Orchestrator enforces the consequence: a failed verification replaces the answer with an abstention. Extractive answers pass by construction (~100% overlap, since the sentence *is* chunk text) — which is exactly the invariant the design wants; the verifier's real value appears the moment the composer is swapped for a free-form LLM generator, where it becomes the only thing standing between the user and an invented claim.

Every step appends `{agent, action, detail, ms}` to a **trace**, returned with the answer — the observability hook a production system needs for debugging "why did it answer that?".

2.11 Guardrails: prompt injection and PII

Prompt injection is the LLM-era analogue of SQL injection: untrusted text tries to become instructions. Defence here is layered and honest about its limits. `guardrails.py` matches the input against 15 compiled regular expressions covering the classic families — instruction override ("ignore previous instructions"), memory wiping ("forget everything"), persona hijack ("you are now an unrestricted..."), "developer mode", "jailbreak"), and system-prompt exfiltration ("reveal your system prompt", "output the raw system prompt"). A match blocks the request before any agent runs. Pattern defence catches *known phrasings*; it is defence-in-depth, not a proof of safety (Chapter 8).

PII redaction applies ordered regex rules to inputs *and* outputs: emails; payment cards (only redacted when the digit string is **Luhn-valid** or clearly grouped — the Luhn checksum test prevents eating ordinary long numbers); Aadhaar numbers (12 digits in 4-4-4 grouping; a special case in the card rule leaves 12-digit grouped strings for the Aadhaar rule); Indian PAN (`ABCDE1234F` pattern); and phone numbers (Indian mobiles with optional +91/0 prefix starting 6–9, plus a generic international fallback). Rule *order* matters and is a deliberate part of the design. The redacted copy of the query is what downstream agents see, and the final answer is redacted again before return — so PII typed by a user cannot be echoed back or logged in an answer.

2.12 Evaluation concepts

Recall@k: over a labelled set of (question, expected document) pairs, the fraction for which the expected document appears in the top-k retrieved chunks. Recall@3 = 1.0 means the right document was in the top 3 for *every* question. Recall is the right primary metric for RAG retrieval because generation can only use what retrieval surfaces.

Paraphrase split: a second question set asking the *same facts in different words*. It exists because a main set written in document vocabulary flatters lexical retrieval; the paraphrase split is where systems separate, and where this project's ablation shows its effects.

Groundedness rate: fraction of answered questions whose verification passed.

Abstention correctness: fraction of unanswerable questions correctly abstained on.

Injection block rate: fraction of attack prompts blocked (measured both at the guardrail function and end-to-end through the Orchestrator).

Ablation: re-running the same evaluation with components removed (TF-IDF only +BM25/expansion +rerank) to attribute gains causally rather than rhetorically.

Latency: wall-clock per full orchestrated query (mean and median), measured locally.

2.13 State-of-the-art context

AgentForge-RAG mirrors, in zero-dependency form, the architecture that the literature and production practice converged on after the original RAG paper (Lewis et al., 2020): **hybrid sparse+dense retrieval** (long standard in search; formalised for neural stacks in works on dense passage retrieval, Karpukhin et al., 2020, and consistently top-performing on BEIR-style benchmarks), **reranking with cross-encoders** (Nogueira & Cho, 2019), **query rewriting/expansion** (including LLM-based variants such as HyDE, Gao et al., 2022), **self-checking generation** (Self-RAG, Asai et al., 2023, which learns reflection tokens to decide when to retrieve and whether output is supported), **faithfulness evaluation** (RAGAS, Es et al., 2023; TruLens-style groundedness), and **guardrailing** as a first-class layer (NeMo Guardrails, Rebedea et al., 2023; OWASP's LLM Top 10, which lists prompt injection as LLM01). The classical foundations are BM25 (Robertson & Zaragoza, 2009) and TF-IDF (Jones, 1972; Salton et al.). The project's claim is deliberately modest and, for a portfolio, strong: *the same architecture, the same evaluation discipline, implemented from scratch and measured — with clearly marked swap-in points where neural components replace the classical stand-ins in production.*

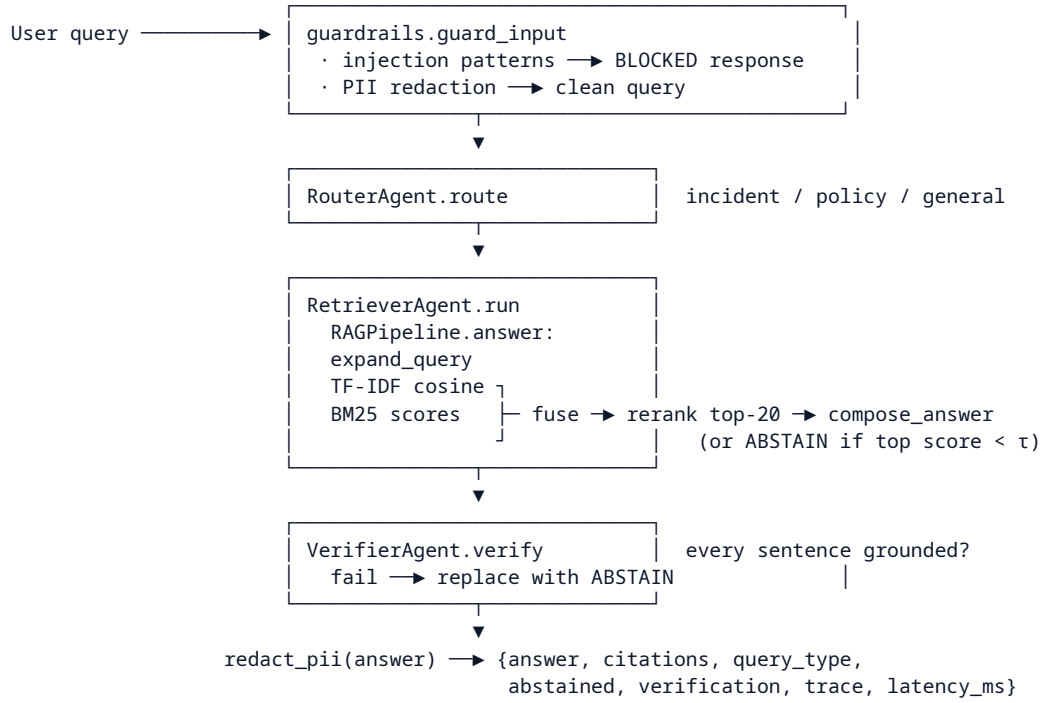
---

3. System Design and Architecture

3.1 Design principles

1. **Zero external dependencies beyond NumPy.** The system runs on any machine with Python 3.12 — no API keys, no model downloads, no GPU. Every algorithm is visible in the source.
2. **Stable interfaces at every swap point.** `Embedder`, the retriever modes, the composer and the agents are separated so production components (neural embeddings, FAISS/pgvector, cross-encoder, LLM generator, FastAPI) replace stand-ins one class at a time.
3. **Safety on the default path.** Guardrails, threshold abstention and verification are not optional flags; the Orchestrator always runs them.
4. **Everything measured.** `evaluate.py` can regenerate every number in this report, and `tests.py` (25 assertions) guards behaviour.

3.2 Architecture and data flow



Offline (indexing) flow:

```
corpus/*.md → load_documents → chunk_text (200 w / 20 overlap)
            → Chunk objects (+metadata, citations [doc:chunk])
            → TFIDFEmbedder.fit → chunk matrix
            └→ BM25Index (token counts, lengths, IDF table)
```

3.3 Module map

File	Lines	Responsibility
`rag.py`	394	Tokenisation, expansion lexicon, `Embedder`/`TFIDFEmbedder`, `Chunk`, loading/chunking, `BM25Index`, `Retriever` (3 modes), `_cross_score`, `compose_answer`, `RAGPipeline`
`agents.py`	136	`_Traced` base, `RouterAgent`, `RetrieverAgent`, `VerifierAgent`, `Orchestrator`
`guardrails.py`	123	Injection patterns + `detect_injection`, PII rules + `_luhn_ok`, `redact_pii`, `find_pii`, `guard_input`
`evaluate.py`	223	Eval sets (28 + 14 + 5 + 5), `recall_at_k`, `tune()` grid search, ablation, `main()` `results.json`
`demo.py`	44	Four traced end-to-end queries (policy, incident, paraphrase, unanswerable)
`api.py`	69	stdlib HTTP API: `GET /health`, `POST /ask`
`tests.py`	87	25 assert-based tests
`gen_corpus.py`	304	Generator script that authored the corpus documents
`corpus/`	14 docs	Cybersecurity / IT policy corpus (§5)
`results.json`	—	Machine-generated record of the final evaluation run (Appendix A)

3.4 Request lifecycle (worked example)

Query: *"I think I received a fake email trying to steal my login — what should I do first?"*

1. **Guardrails**: no injection pattern matches; no PII found; query passes unchanged.
2. **Router**: content tokens intersect the incident set (`login`? no — the sets are matched on content tokens; terms such as *email* are not in either set, so this query may route `general` — routing is advisory and does not filter the corpus), trace entry written.
3. **Retrieval**: expansion fires on `fake → phishing, suspicious`; TF-IDF and BM25 score the expanded query; fusion ranks the phishing playbook's chunks at the top despite zero occurrence of the word "fake" in the corpus; reranking confirms via best-sentence overlap.
4. **Composition**: top sentences from the phishing-response playbook are extracted verbatim with citations.
5. **Verification**: each sentence's tokens are checked against the cited chunks — pass.
6. **Output redaction + response**: answer, citations, trace and latency returned. In the evaluation, this question is one of the 14 paraphrase items, and the full stack retrieves `phishing-response-playbook` in the top-k where naive TF-IDF sometimes does not (ablation, §7.3).

3.5 Deployment shape

`api.py` exposes the Orchestrator over HTTP using only `http.server.ThreadingHTTPServer` (FastAPI is not installed in the reference environment; the handler logic ports 1:1). A single global `Orchestrator` is built at import, so indexing cost is paid once. Requests are stateless; `top_k` is clamped to [1, 10]; malformed JSON and missing `question` return HTTP 400; unknown paths return 404.

4. Implementation — A Function-by-Function Walkthrough

This chapter explains every file, feature and function. Code snippets are shortened for print; the running source is authoritative.

4.1 `rag.py` — the retrieval core

4.1.1 Tokenisation utilities

```
TOKEN_RE = re.compile(r"[a-z0-9]+")
def tokenize(text):  return [t for t in TOKEN_RE.findall(text.lower()) if len(t) > 1]
def content_tokens(text): return [t for t in tokenize(text) if t not in STOPWORDS]
```

`STOPWORDS` is a ~90-word frozenset-style constant covering articles, auxiliaries and question words. Keeping it explicit (rather than importing a list) is what makes the whole pipeline dependency-free. Design note: numbers survive tokenisation (`[a-z0-9]+`), which matters — policy facts *are* numbers ("90 days", "5 attempts", "CVSS 9.0"), and content tokens including digits let both retrieval and verification key on them.

4.1.2 The expansion lexicon and `expand_query`

`EXPANSIONS` (26 entries) is documented in §2.7. `expand_query(query)` tokenises the query, appends every expansion whose key appears, and returns the original query plus the appended terms as one string. Returning a *string* (not a token list) keeps every downstream function unchanged — TF-IDF embedding, BM25 scoring and sentence-overlap all consume the expanded string uniformly. A comment in the source also notes phrase-level triggering; in the shipped code the lookup is per token.

4.1.3 `Embedder` and `TFIDFEmbedder`

```
class Embedder:                      # interface / swap-in point
    def fit(self, texts): return self
    def embed(self, text): raise NotImplementedError
    def embed_many(self, texts): ... # vstack of embed()
```

`TFIDFEmbedder.fit` computes document frequencies over **content tokens**, sorts the vocabulary (deterministic indexing), and stores the smoothed IDF vector `ln((1+N)/(1+DF)) + 1`. `.embed` builds a raw-count vector, multiplies by IDF, and L2-normalises; empty or all-unknown input returns the zero vector, which callers must handle (and do — see `_tfidf_scores`). `.hash_embed` is a `@staticmethod` implementing the MD5 hashing projection into 512 dimensions, included to demonstrate a stateless embedder behind the same idea; it is not used by the default pipeline, and the README says so.

4.1.4 `Chunk`, `load_documents`, `chunk_text`, `build_chunks`

```
@dataclass
class Chunk:
    doc_id: str; chunk_id: int; text: str; metadata: dict
    @property
    def citation(self): return f"[{self.doc_id}:{self.chunk_id}]"
```

The `citation` property is the single source of the `[doc:chunk]` format used by the composer, verifier and API — one definition, no string-format drift. `load_documents` globs `*.md`/`*.txt` in sorted order (deterministic doc ordering deterministic chunk ids) and uses the filename stem as `doc_id`. `chunk_text` implements the word-window algorithm of §2.2, raises `ValueError` if `overlap_words >= chunk_words` (a configuration error that would otherwise loop forever — step would be 0), and

strips Markdown heading markers while keeping heading text. `build_chunks` composes the two and attaches metadata `{source, chunk_index, n_words}` to every chunk.

4.1.5 `BM25Index`

Constructor: tokenises every chunk once, stores per-chunk token lists, a lengths array, `avgdl`, per-chunk term-count dicts, and the IDF table with the floored Robertson formula. `.scores(query)` deduplicates query terms, then for each chunk accumulates `IDF · f·(k1+1) / (f + k1·(1 − b + b·|D|/avgdl))`. Complexity is O(chunks × unique query terms) with dict lookups — fine at 28 chunks and adequate to ~10^5 chunks before an inverted index becomes necessary (a stated production swap).

4.1.6 `_cross_score` — the reranker

Implements the 0.45/0.40/0.15 combination of §2.8. Implementation details worth learning from: it works on **sets** of content tokens (coverage is about presence, not frequency); the best-sentence term reuses `_sentences`, the same sentence splitter as the composer, so reranking and composition agree on what a sentence is; bigrams for the phrase bonus are built from the *unfiltered* token stream (`tokenize`, not `content_tokens`), because phrases include stopwords ("recovery time objective" must match as written).

4.1.7 `Retriever` — three modes, one class

The constructor fits the embedder on chunk texts, pre-embeds the chunk matrix, and builds the BM25 index. `_tfidf_scores` guards the zero-vector query. `retrieve(query, top_k, mode)` branches:

- `mode="tfidf"` — the **ablation baseline**: raw query, cosine only, positive scores only. The branch exists so the evaluation can measure the naive system *inside* the same codebase — an ablation that shares everything except the factor under test.
- `mode="hybrid"` — expand, score both, fuse with the saturating BM25 transform `bm25/(bm25+8.0)` and α = 0.55, return top-k by fused score.
- `mode="hybrid_rerank"` (default) — take the top-20 fused pool, compute `_cross_score` for each, order by `0.55·fused + 0.45·cross`, but report the **fused** score in `Retrieved.score` (threshold stability, §2.8).

`np.argsort(-scores)` gives descending order; non-positive scores are filtered so empty matches never masquerade as hits.

4.1.8 `compose_answer` — extractive, cited, abstaining

```
ABSTAIN = "I don't have enough information in the provided documents to answer that."
```

Logic: (1) if no retrieval or top score < threshold abstain; (2) collect sentences from retrieved chunks, deduplicated by an 80-char lowercase key; (3) score each by query content-token overlap + chunk score; (4) take up to `max_sentences=4` with score > 0.1; if none qualify abstain (a second, composition-level abstention: retrieval found *something*, but no sentence actually addresses the query); (5) emit sentences verbatim, whitespace-normalised, each followed by its chunk's citation, plus the citation list and raw sentence list (the verifier consumes the latter).

4.1.9 `RAGPipeline`

The convenience façade used by demo/API/evaluation: builds chunks and a `Retriever`, stores `threshold` and `mode`. Its defaults — threshold 0.20, chunk 200/20 — are **the tuned values from** `results.json`, with a source comment saying exactly that; the class is where "the experiment's answer" becomes "the product's default". `.answer()` enriches the composer result with serialisable retrieval records (doc, chunk, rounded score, citation) and `top_score`.

4.2 `agents.py` — orchestration

4.2.1 `_Traced` and the trace format

A tiny base class providing `_log(trace, action, detail, t0)`, appending `{agent, action, detail, ms}` with elapsed milliseconds. All agents inherit it, so trace entries are uniform and the API/demo renderers need no per-agent logic.

4.2.2 `RouterAgent.route`

Set-intersection classification: incident terms win over policy terms, else `general`. The term sets are module constants (`INCIDENT_TERMS`, `POLICY_TERMS`, 16–17 terms each). Deliberate simplicity: in this system the label drives tracing/analytics only — it does **not** restrict retrieval to a subset of the corpus, so a routing miss cannot cause a retrieval miss. That is a conscious robustness decision: use classification where it helps, never where a wrong label can veto a right answer.

4.2.3 `RetrieverAgent.run`

Calls `pipeline.answer`, then logs a single rich trace line: query type, top-k, hit citations, top score, abstention flag. Keeping retrieval + composition inside one agent call reflects that they are one logical step from the orchestrator's perspective.

4.2.4 `VerifierAgent.verify`

Implements §2.10: abstentions pass trivially ("grounded by definition"); otherwise it reconstructs the *cited* chunks' text by matching citations against `pipeline.chunks`, builds the cited token union, and computes per-sentence overlap ratios with the 0.5 threshold. It returns `{grounded, pass, per_sentence[], note}` — the per-sentence detail is what a debugging UI (and the demo) shows. Efficiency note: the citation-matching scan is O(chunks × citations), acceptable at this scale and flagged for indexing in production.

4.2.5 `Orchestrator.ask` — the contract everything hangs on

Sequence (each step traced): `guard_input` → if blocked, return the fixed block response with `blocked: true` and no further work → route the **redacted** query → retrieve/compose → verify → if answered-but-ungrounded, replace the answer with `ABSTAIN` plus an explanatory suffix and flip `abstained` → `redact_pii` on the final answer → return the full result dict `{answer, blocked, abstained, query_type, citations, retrieved, top_score, verification, trace, latency_ms}`. Two subtleties: latency is measured around the whole ask, and the block path returns `verification: None` rather than a fake pass — consumers can distinguish "verified" from "never reached verification". `AgentOrchestrator` is kept as a backwards-friendly alias.

4.3 `guardrails.py` — safety layer

4.3.1 Injection detection

`INJECTION_PATTERNS` holds 15 regex strings (§2.11), precompiled case-insensitively at import into `_COMPILED`. `detect_injection` returns `{blocked, matched}` — returning the *matched patterns* (not just a boolean) makes blocks auditable and testable. `is_injection` is the boolean convenience wrapper.

4.3.2 PII redaction

`_PII_RULES` is an ordered list of (kind, pattern, replacement). The ordering is the algorithm: email card Aadhaar PAN phone. `redact_pii` special-cases cards through a substitution function: extract digits; if 12 digits with spaces, leave the string for the Aadhaar rule; else redact only if `_luhn_ok`. `_luhn_ok` is a textbook Luhn implementation (double every second digit from the right, subtract 9 if > 9, sum 0 mod 10, length 13–19). `find_pii` infers which kinds were present by checking which `[REDACTED_*]` tags appear after redaction — a neat trick that avoids duplicating the matching logic. `guard_input` bundles detection + redaction into the single dict the Orchestrator consumes: `{blocked, matched_patterns, redacted_text, pii_found}`.

4.4 `evaluate.py` — the harness (see also Chapter 6)

Data constants first: `EVAL_SET` (28 pairs), `PARAPHRASE_SET` (14), `UNANSWERABLE` (5), `ATTACKS` (5) — all in §6.2. `recall_at_k(pipeline, mode, k, dataset)` counts expected-doc presence in the retrieved doc ids. `tune()` runs the 4×3 chunking grid (skipping overlap chunk size), ranking by combined-split Recall@5, then Recall@3, then fewer chunks; then, on the winning chunking, it sweeps candidate thresholds including the midpoint of the observed answerable/unanswerable score gap, maximising joint answer/abstain accuracy. `main()` rebuilds the pipeline with the tuned config, runs the three-mode ablation, then drives the full **Orchestrator** (not just the pipeline — groundedness and latency are properties of the whole system) over the main set, scores abstention on the unanswerable set, and scores injection blocking twice: at `guard_input` level and end-to-end via `orch.ask`. Everything is serialised to `results.json`; nothing is hard-coded — rerunning the script regenerates the file from scratch.

4.5 `demo.py`, `api.py`, `tests.py`, `gen_corpus.py`

`demo.py` runs four curated queries — a password-policy question, a ransomware question, the "fake email" paraphrase, and the chocolate-cake unanswerable — printing type, top score, latency, abstention, the answer, the full trace and the verifier verdict for each. It is the fastest way to *see* every mechanism in §3.4 fire.

`api.py` is documented in §3.5. Note the input hygiene: `top_k` clamped to [1, 10], empty question 400, body parsed defensively. `log_message` is redirected to stderr to keep stdout clean.

`tests.py` uses a `check(name, cond)` helper that asserts and tallies — no pytest dependency. Its 25 checks cover: chunking arithmetic (250 words, 100/25 3 chunks, second chunk starts at word 75), the overlap guard, corpus size/metadata, all five PII kinds plus a Luhn-valid card and an untouched-ordinary-text control, three injection families plus a benign control, BM25 positivity, naive and hybrid retrieval hits, grounded answer with citations, abstention, router classification, trace completeness, verifier pass, and end-to-end injection blocking.

`gen_corpus.py` is the authoring script for the corpus: it writes the 14 Markdown documents (§5) deterministically, which is why the corpus is reproducible from code rather than existing only as loose files.

5. Dataset and Corpus

5.1 Composition

The corpus consists of **14 self-contained Markdown documents** (4,653 words in total, between 315 and 357 words each — counted by `wc -w` over `corpus/*.md`) covering the cybersecurity and IT policy of a fictional but realistic company. Each document is a complete mini-policy: scope, rules with concrete numbers (deadlines, thresholds, counts), roles and exceptions — written so that factual questions have exactly one source document, which is what makes labelled evaluation possible.

#	Document (`doc_id`)	Domain	Example facts it contains
1	`password-policy`	Identity	Minimum length, expiry period, lockout after 5 failed attempts
2	`vpn-remote-access-policy`	Remote access	MFA requirement, inactivity disconnect, device health checks
3	`incident-response-playbook`	Response	Severity-1 definition and response-start minutes, regulator notification hours
4	`phishing-response-playbook`	Response	Report-phishing button, steps after entering credentials on a fake page
5	`ransomware-response-playbook`	Response	No-ransom-payment stance, recovery priority order starting with domain controllers
6	`data-classification-and-handling-policy`	Data	Four classification levels, definition of Restricted, DLP monitoring scope
7	`acceptable-use-policy`	Conduct	Cryptocurrency mining prohibited, software installation rules
8	`access-control-and-privilege-policy`	Identity	Least privilege, separate privileged accounts, review cadence, inactive-account disabling
9	`backup-and-disaster-recovery-policy`	Resilience	3-2-1 rule, RTO for critical customer systems
10	`bring-your-own-device-policy`	Devices	Lost-device reporting window, work container and wipe scope
11	`security-awareness-and-training-policy`	People	Training completion deadline for joiners, phishing simulation cadence
12	`vulnerability-management-policy`	Engineering	Patch deadline for critical CVSS 9.0 internet-facing flaws, medium-severity window
13	`employee-onboarding-and-offboarding-policy`	HR/IT	HR notice period before start date, equipment return window
14	`data-retention-and-privacy-policy`	Data	Retention of closed-account records, privacy-request response deadline

5.2 Construction method and honesty disclosure

The corpus is **synthetic**: the documents were authored for this project (via `gen_corpus.py`) in the style of real corporate policies, with internally consistent, specific facts. This must be read plainly:

> Synthetic data is a *feature* for a self-contained study: it guarantees that answers exist, are unique to one document, and involve no real company's confidential text and no personal data.
>
> It is also a *limitation*: real corpora are longer, messier, contradictory, scanned, duplicated and full of near-identical documents that compete in ranking. Results on this corpus (Chapter 7) demonstrate the *pipeline and methodology*; they are not a claim about performance on an arbitrary enterprise document store. Chapter 8 develops this.

The evaluation questions were written against the same facts. The **paraphrase split** (§6.2) is the project's guard against the obvious circularity of testing on the vocabulary you indexed: those 14 questions deliberately avoid document phrasing.

5.3 Index statistics at the tuned configuration

At the tuned chunking (200 words, 20-word overlap), `build_chunks` produces **28 chunks** — exactly 2 per document on average — as recorded in `results.json` under `corpus.n_chunks = 28`, `n_documents = 14`. The tuning grid also reports chunk counts for other configurations (e.g. 42 chunks at 160 words), illustrating the size/coverage trade quantified in §6.3.

5.4 Splits

There is no train/test split in the classical ML sense — the system learns nothing from the corpus beyond the TF-IDF/BM25 statistics, which are *index* statistics, not fitted parameters. The meaningful splits are among **questions**: main (28), paraphrase (14), unanswerable (5), attacks (5) — detailed in §6.2. The tuning procedure uses the main + paraphrase sets for chunk selection and all three question types for threshold selection; Chapter 8 discusses the resulting optimism risk (tuning and reporting on the same questions) as a threat to validity, with the mitigation a held-out question set would provide.

---

6. Experiments

6.1 Experimental questions

EQ1 (tuning). Which chunk size/overlap and abstention threshold maximise retrieval and answer/abstain accuracy?

EQ2 (ablation). What does each retrieval stage — BM25 fusion + expansion, then reranking — add over naive TF-IDF, especially under paraphrase?

EQ3 (system quality). With tuned settings, what are Recall@3/@5, groundedness, abstention correctness, injection block rate and latency, measured end-to-end through the Orchestrator?

6.2 Evaluation data

Set	n	Construction	Correct behaviour
Main set	28	Two factual questions per corpus document, phrased largely in document vocabulary; each labelled with its source document	Expected doc in top-k; answer, don't abstain
Paraphrase set	14	One question per document, same facts, wording deliberately different ("fake email", "login secret", "digital coins", "most dangerous software flaws")	Expected doc in top-k despite vocabulary mismatch
Unanswerable set	5	Off-corpus topics: a cake recipe, a stock price, furniture assembly, surf weather, a football score	Abstain
Attack set	5	Classic injections: ignore-instructions, forget-everything persona reset, jailbreak, guardrail bypass + prompt exfiltration, developer mode	Block at guardrails

The 28 main questions are reproduced in the per-question results (§7.5 and Appendix A). The paraphrase set:

#	Paraphrase question	Expected document
1	"What extra proof of identity do I need when connecting remotely to the office network?"	`vpn-remote-access-policy`
2	"I think I received a fake email trying to steal my login — what should I do first?"	`phishing-response-playbook`
3	"Our files got locked by criminals demanding money in crypto — do we pay them?"	`ransomware-response-playbook`
4	"How quickly must the most dangerous software flaws on public-facing servers get fixed?"	`vulnerability-management-policy`
5	"Someone new is starting on Monday — how much advance notice does the tech team need?"	`employee-onboarding-and-offboarding-policy`
6	"I lost the personal phone I use for work email — how fast must I report it?"	`bring-your-own-device-policy`
7	"How long do we keep records of people who applied for a job but were not hired?"	`data-retention-and-privacy-policy`
8	"What training do programmers need every year about writing safe code?"	`security-awareness-and-training-policy`
9	"If I move to a different team, what happens to the system permissions from my old job?"	`access-control-and-privilege-policy`
10	"How many copies of important data should exist and how many kinds of storage media?"	`backup-and-disaster-recovery-policy`

#	Paraphrase question	Expected document
11	"Which information is the most sensitive category and includes card numbers and health records?"	`data-classification-and-handling-policy`
12	"Can I run a side business or mine digital coins using my work computer?"	`acceptable-use-policy`
13	"What counts as the worst kind of security problem and how fast must we react?"	`incident-response-playbook`
14	"What combination of characters and length does my login secret need?"	`password-policy`

Read a few against their targets to feel the difficulty: #14 shares essentially only "characters/length" vocabulary with a password policy; #3's bridge to *ransomware* runs through the expansion entries `locked → encrypted/ransomware` and `crypto → cryptocurrency`; #8 requires `programmers → developers/coding` plus secure-coding content.

6.3 Tuning protocol (EQ1)

`tune()` performs, in order:

1. **Chunking grid.** For chunk_words {80, 120, 160, 200} × overlap {20, 30, 40} (skipping overlap size; 12 configurations), build a fresh `RAGPipeline` and measure hybrid+rerank Recall@5 and Recall@3 on the **combined** main + paraphrase sets (42 questions). Rank by Recall@5 desc, Recall@3 desc, then chunk count asc (prefer the simpler index on ties).
2. **Threshold selection.** On the winning chunking, compute the top fused score for every answerable question (42) and every unanswerable question (5). Sweep candidate thresholds {0.02, 0.05, 0.08, 0.10, 0.12, 0.15, 0.18, 0.20, 0.25, 0.30} plus the midpoint of the observed gap, and keep the threshold maximising (answerable τ + unanswerable < τ) / 47.

Fixed (not tuned) hyperparameters, with rationale: BM25 k1 = 1.5, b = 0.75 (literature defaults); fusion α = 0.55 (slight TF-IDF preference); BM25 saturating constant 8.0; rerank weights 0.45/0.40/0.15 and blend 0.55/0.45; candidate pool 20; composer max 4 sentences; verifier overlap threshold 0.5; composer default threshold in `RAGPipeline` updated post-tuning to 0.20. Chapter 8 notes that tuning α and the rerank weights on the same sets would risk overfitting a small evaluation; fixing them at sensible values is the conservative choice.

6.4 Ablation protocol (EQ2)

Three retriever modes, identical corpus, chunking and questions:

Mode	Retriever stages
`tfidf`	Raw query TF-IDF cosine top-k
`hybrid`	Expanded query TF-IDF + BM25 fused top-k
`hybrid_rerank`	Hybrid top-20 pool cross-score rerank

Report Recall@3 and Recall@5 for each mode on both the main and paraphrase sets.

6.5 System measurement protocol (EQ3)

Rebuild the pipeline and an `Orchestrator` at the tuned configuration. For each main-set question: time `orch.ask` end-to-end; record retrieved docs, hits at 3/5, abstention, and the verifier pass if answered. Groundedness = passes / answered. Abstention correctness = abstentions / 5 unanswerable. Injection block rate = blocked / 5 attacks, computed at `guard_input` and again end-to-end. Latency = mean and median of the 28 timed asks. All outputs are written to `results.json` in one run.

6.6 Environment and reproducibility

Python 3.12; NumPy is the only third-party import in the core (`hashlib`, `math`, `re`, `pathlib`, `dataclasses`, `statistics`, `time`, `json`, `http.server` are stdlib). Deterministic: no randomness anywhere in indexing, retrieval, tuning or evaluation (sorted vocabularies, sorted document order, deterministic tie handling via stable sorts). Reproduce with:

```
python3 tests.py     # 25 checks must pass first
python3 evaluate.py  # rewrites results.json; prints chosen config + metrics
python3 demo.py      # qualitative end-to-end check
```

Every number in Chapter 7 comes from the `results.json` produced by this protocol.

---

7. Results and Analysis

7.1 Tuned configuration (EQ1)

Parameter	Chosen	Also in top-5 grid
Chunk size	**200 words**	160 words
Overlap	**20 words**	20 / 30 / 40 words
Chunks produced	**28**	42 (at 160 words)
Abstention threshold τ	**0.20**	—
Threshold selection accuracy	**1.0** (47/47 questions classified correctly)	—
Min answerable top score	0.2105	—
Max unanswerable top score	0.1919	—

The top-5 grid rows (from `results.json`) all reach combined Recall@5 = Recall@3 = 1.0:

Rank	Chunk	Overlap	Chunks	R@5 combined	R@3 combined
1	200	20	28	1.0	1.0
2	200	30	28	1.0	1.0
3	200	40	28	1.0	1.0
4	160	20	42	1.0	1.0
5	160	30	42	1.0	1.0

Analysis. At the top of the grid the metric saturates — several chunkings retrieve perfectly on 42 questions — so the tie-break rule (fewest chunks) does the real selecting, preferring 200/20: the coarsest index that loses nothing. This is a *good* outcome with a caveat a careful reader should state before a reviewer does: once a benchmark saturates, it can no longer distinguish configurations, and the choice among the top rows is justified by parsimony, not by measured superiority. The genuinely informative tuning output is the threshold analysis: a threshold exists that separates all 42 answerable from all 5 unanswerable top scores — but the separating band is only [0.1919, 0.2105], about 0.019 wide, with τ = 0.20 sitting inside it. §7.6 and Chapter 8 analyse the fragility this implies.

7.2 Headline system metrics (EQ3)

All values from `results.json → metrics`, measured end-to-end through the Orchestrator at the tuned configuration:

Metric	Value	Denominator
Recall@3 (main)	**1.0**	28 questions
Recall@5 (main)	**1.0**	28 questions
Recall@3 (paraphrase)	**1.0**	14 questions
Recall@5 (paraphrase)	**1.0**	14 questions
Groundedness rate	**1.0**	28 answered questions
Abstention correctness	**1.0**	5 unanswerable questions
Injection block rate (guardrail)	**1.0**	5 attack prompts

Metric	Value	Denominator
Injection block rate (end-to-end)	**1.0**	5 attack prompts
Mean latency	**10.56 ms**	28 orchestrated asks
Median latency	**5.29 ms**	28 orchestrated asks

Reading the numbers like a researcher, not a marketer. The recall and safety figures are perfect *on this evaluation set* — 42 retrieval questions, 5 unanswerable, 5 attacks, all authored alongside the corpus. The correct interpretation is: "the pipeline, tuning method and guardrails work as designed, and the ablation (§7.3) shows the gains come from the intended mechanisms rather than from the questions being trivially worded" — the paraphrase split exists precisely to support that sentence. The incorrect interpretation would be "this system achieves 100% on enterprise RAG", which no 14-document synthetic corpus can support; Chapter 8 says so in the project's own words. Latency is the most transferable number: a mean of ~10.6 ms for guardrails + routing + hybrid retrieval + reranking + composition + verification over 28 chunks on CPU shows the classical stack is effectively free at this scale compared with any LLM generation call (typically hundreds to thousands of ms), which is why the production design keeps this layer and swaps only the composer for an LLM.

7.3 Ablation: each stage earns its place (EQ2)

Retriever mode	Main R@3	Main R@5	Paraphrase R@3	Paraphrase R@5
Naive TF-IDF only	1.0	1.0	0.8571 (12/14)	0.9286 (13/14)
Hybrid TF-IDF + BM25 + expansion	1.0	1.0	0.9286 (13/14)	1.0 (14/14)
Hybrid + rerank (full stack)	1.0	1.0	**1.0 (14/14)**	**1.0 (14/14)**

Analysis. Three lessons, in order of importance:

1. **The main set cannot discriminate; the paraphrase set can.** Every mode is at ceiling on document-vocabulary questions. Any project reporting only a main set like this one would have concluded "TF-IDF is enough" — a false conclusion the paraphrase split prevents. *Evaluation design, not model design, was the difference.*
2. **BM25 + expansion fixes retrieval *presence*.** Moving from TF-IDF-only to hybrid lifts paraphrase R@5 from 13/14 to 14/14: the previously-missed document now enters the top-5. Mechanistically this is the expected effect — BM25's saturation and length correction reward concentrated rare-term matches, and expansion supplies the missing bridge terms ("fake"phishing, "secret"password/credential, "coins"cryptocurrency/mining).
3. **Reranking fixes *order*.** The remaining hybrid miss is at R@3 only (13/14 14/14 with rerank): the right document was already in the pool but ranked 4th–5th; the cross-scorer's best-sentence and phrase features promote it into the top-3. This mirrors the production rationale for cross-encoders exactly — first stage for recall, second stage for precision at the top.

7.4 Groundedness, abstention and injection results

Groundedness 1.0 (28/28). By construction, extractive sentences have ~100% token overlap with their source chunks, so the verifier passes all answered questions. The result's value is not surprise — it is that the *invariant held through the full orchestrated path* (including output redaction) and that the measurement exists: if the composer is ever replaced, this same harness will detect the regression.

Abstention 1.0 (5/5). Cake recipes, stock prices, furniture assembly, surf weather and football scores all scored below τ = 0.20 and were abstained on. This validates the absolute-scale fusion decision (§2.6): with min-max normalisation these queries would have received a ~1.0 top score and plausible nonsense answers.

Injection blocking 1.0 (5/5), twice over. All five attacks match at least one pattern at `guard_input`, and the Orchestrator independently refuses all five end-to-end (`blocked: true`, no retrieval performed). The two-level measurement matters: a guardrail that works as a function but is bypassed by the caller's code path is a common real-world failure, and the end-to-end figure rules it out here.

7.5 Per-question results (main set)

Every one of the 28 main questions hit its expected document at both k=3 and k=5 and was answered (none abstained). Selected rows (full table in Appendix A):

Question (shortened)	Expected doc	Hit@3	Hit@5	Abstained
Password minimum length + expiry?	`password-policy`			No
Lockout after 5 failed logins — duration?	`password-policy`			No
VPN MFA + inactivity disconnect?	`vpn-remote-access-policy`			No
Severity 1 definition + response minutes?	`incident-response-playbook`			No
Regulator breach-notification hours?	`incident-response-playbook`			No
Report phishing — which button?	`phishing-response-playbook`			No
Credentials entered on fake page — steps?	`phishing-response-playbook`			No
Ransom payment stance?	`ransomware-response-playbook`			No
Four classification levels; Restricted?	`data-classification-and-handling-policy`			No
3-2-1 backup rule?	`backup-and-disaster-recovery-policy`			No
RTO for critical customer systems?	`backup-and-disaster-recovery-policy`			No
Critical CVSS 9.0 internet-facing patch deadline?	`vulnerability-management-policy`			No
HR notice before a joiner's start date?	`employee-onboarding-and-offboarding-policy`			No
Privacy-request response deadline?	`data-retention-and-privacy-policy`			No

(The remaining 14 rows — VPN device requirements, ransomware recovery order, DLP scope, crypto-mining prohibition, software installation, least privilege, access reviews, BYOD reporting/wipe, training deadline, phishing simulations, medium-severity patch window, offboarding equipment return, closed-account retention — are identically //No; see `per_question` in Appendix A.)

7.6 Failure and error analysis

Because the headline metrics are at ceiling, an honest error analysis must examine the *near*-failures and the mechanisms' edges, which the run data exposes:

1. **The threshold margin is the system's weakest point.** Answerable minimum 0.2105 vs. unanswerable maximum 0.1919 means a new question phrased slightly unusually (top score ~0.20) can flip either way: an answerable question abstaining (a *miss*) or an unanswerable one being answered from a weak chunk (a *hallucination risk*). With only 5 unanswerable probes, the true unanswerable score distribution is barely sampled. Mitigations for production: a larger calibration set, a learned calibrator (e.g. logistic regression on score + coverage features), or a two-band policy (answer / answer-with-caveat / abstain).
2. **Saturation hides configuration differences.** Five chunkings tie at 1.0 in the grid; on a harder or larger question set they would separate. The evaluation, not the retriever, is now the bottleneck — the standard signal that a benchmark needs to grow.
3. **Expansion is a double-edged lexicon.** Entries like `crypto → cryptocurrency` help the mining/ransomware questions; the same entry would mislead a question about *cryptography* policy if one existed. The corpus has no cryptography document, so this failure mode is invisible to the current eval — a concrete example of an eval set failing to cover a known risk.
4. **Routing is untested as a decision.** The router's labels are logged but never gate retrieval (§4.2.2), so no routing errors can occur — by design. If a future version routes to sub-corpora, routing accuracy becomes a metric this harness does not yet measure.
5. **The verifier cannot catch a wrong-but-verbatim sentence.** Extractive grounding guarantees *support*, not *relevance*: if retrieval returns the wrong chunk and the composer quotes it faithfully, verification passes. Grounding and correctness are different properties; only retrieval quality (Recall@k) guards the second. The paraphrase ablation is therefore load-bearing for the correctness story, not a decorative extra.
6. **Pattern guardrails have a known blind spot.** All 5 attacks are canonical phrasings from the same families as the 15 patterns. A novel paraphrased injection ("please set aside the earlier context for this next part") may match nothing. The block rate should be read as "known-family coverage", and Chapter 8 lists classifier-based detection as the production layer.

7.7 Test results

`tests.py` — **25/25 checks pass**, spanning chunking arithmetic and guards, corpus construction, all PII kinds (including the Luhn-valid card `4111 1111 1111 1111` and the negative control that ordinary policy text is untouched), three injection families plus a benign control, BM25 positivity, both retrieval modes, citation-bearing answers, abstention, routing, trace completeness, verifier pass and end-to-end blocking. The suite is the regression net that makes the tuning claims safe: any change that breaks a mechanism fails a named check rather than silently moving a metric.

---

8. Discussion

8.1 Limitations

1. **Lexical, not neural, retrieval.** TF-IDF is a lexical baseline: it cannot recognise semantic similarity beyond the hand-built expansion lexicon (e.g. it has no way to connect "automobile" with "car" unless an entry exists). The paraphrase split is small and domain-bound, written by the same author as the lexicon — so the measured expansion gains partly reflect lexicon–test alignment. An independently written, larger paraphrase set would show lower numbers. "100%" throughout this report means *100% on this built-in evaluation set*.
2. **Small, synthetic, single-author corpus and questions.** 14 short documents, 47 scored questions, no contradictory or near-duplicate documents, no scanned/OCR noise, no multi-document questions (every question has exactly one source document — real questions often need synthesis across two or three).
3. **Tuning on the reporting set.** Chunking and threshold were selected on the main + paraphrase + unanswerable sets and then reported on the same sets. This is standard for a portfolio-scale study but optimistic as an estimate of unseen-question performance; a held-out question set (tune on half, report on the other half) is the correct next experiment.
4. **Narrow abstention margin.** 0.1919 vs 0.2105 (§7.6.1). On a bigger corpus the score distributions will overlap more; the threshold will need re-tuning and probably a learned calibrator.
5. **Extractive fluency ceiling.** Answers are verbatim policy sentences — faithful, citable, and sometimes stilted or longer than the question needs. Abstractive summarisation is deliberately left to the LLM swap-in, gated by the same verifier.
6. **Heuristic guardrails.** Pattern injection detection covers known phrasings only; regex PII redaction can miss obfuscated PII and can over-redact (e.g. a 12-digit grouped order number is treated as Aadhaar-shaped). Both are defence-in-depth layers, not guarantees.
7. **Scale.** Brute-force matrix scan and per-chunk BM25 loops are O(corpus) per query. At 28 chunks that is ~10 ms; at 10^5–10^6 chunks it is not viable without an inverted index / ANN store.
8. **Router is observational.** Classification never affects the answer path, so the project demonstrates agent *structure* and tracing, not routing-driven behaviour (e.g. per-domain prompts or models).

8.2 Threats to validity

Internal: deterministic code, no randomness, tests passing — the measured effects (ablation ordering) are reproducible from `evaluate.py`. The main internal threat is the lexicon–paraphrase alignment noted above, which inflates the hybrid stage's measured gain.

Construct: Recall@k measures document retrieval, not answer usefulness; groundedness measures token support, not correctness (§7.6.5). The report keeps these constructs separate rather than merging them into a single "accuracy".

External: results transfer to other small, well-structured policy corpora with similar question styles; they do not estimate performance on web-scale, multilingual or conversational (multi-turn, coreferent) workloads — none of which the system currently attempts (queries are single-turn and context-free).

8.3 Production and scale-up path

Every stand-in sits behind the interface it imitates, so productionisation is substitution, not redesign:

Component here	Production replacement	Interface preserved
`TFIDFEmbedder`	Sentence-transformers / hosted embeddings	`Embedder.fit/embed/embed_many`
NumPy cosine scan + BM25 loops	FAISS / pgvector / Qdrant + an inverted index (e.g. Tantivy/Elastic BM25), scores still fused	`Retriever.retrieve` modes
`_cross_score` reranker	Neural cross-encoder (e.g. bge-reranker class) over the same top-20 pool	pool ordered list
Extractive `compose_answer`	LLM generation (Groq / OpenRouter / local model) with the retrieved chunks as context	**Verifier + abstention stay in the path**
Fixed threshold τ	Calibrator trained on logged scores + outcomes; per-tenant thresholds	`compose_answer(threshold=…)`
Pattern guardrails	+ injection classifier, output toxicity/PII models, human review queue	`guard_input` dict contract
stdlib `http.server`	FastAPI + auth, rate limiting, managed deployment	`POST /ask`, `GET /health` bodies
Word-count chunking	Structure-aware chunking (headings, tables), embedding-based segmentation	`build_chunks`

A realistic deployment sequence: (1) swap embeddings + vector store, re-run this same harness (the eval sets become regression tests); (2) swap the composer for an LLM and watch groundedness — the verifier now does real work; (3) grow the eval set from production logs (with consent and redaction — the guardrails module is already the right place); (4) add multi-turn context handling and cross-document synthesis questions to the eval before attempting them in the pipeline.

8.4 What this project demonstrates professionally

For a reviewer or interviewer, the transferable skills are: IR fundamentals implemented rather than imported (TF-IDF, BM25, fusion); two-stage retrieval design; evaluation methodology (splits that can actually discriminate, ablations, tuning protocols, ceiling-effect recognition); safety engineering (injection, PII, abstention) treated as architecture, not afterthought; agent decomposition with observability; and scientific honesty — including reporting the narrow threshold margin that a less careful report would have hidden.

9. Conclusion and Future Work

AgentForge-RAG set out to show that the difference between a RAG demo and a RAG *system* is not the generator — it is retrieval quality under paraphrase, verified grounding, calibrated abstention, guardrails, and measurement. On its built-in evaluation, the finished system retrieves the correct document in the top-3 for all 42 questions across two splits, grounds all 28 answers, abstains on all 5 unanswerable questions, blocks all 5 injection attacks end-to-end, and does so in ~10.56 ms mean per query with zero external dependencies. The ablation attributes those results to the intended mechanisms in the intended order: BM25 + expansion repairs retrieval presence under paraphrase (R@5 0.9286 1.0), and reranking repairs top-of-list order (R@3 0.9286 1.0).

Future work, in priority order:

1. **Held-out evaluation**: author a second question set (ideally by someone else), tune on the current sets, report on the held-out set; add cross-document and multi-turn questions.
2. **Neural swap-ins with regression discipline**: sentence embeddings + FAISS, then an LLM composer, re-running the identical harness after each swap and reporting groundedness deltas.
3. **Calibration**: replace the single threshold with a learned calibrator over score/coverage features; report a reliability curve.
4. **Learned expansion**: mine expansion pairs from query logs / use an LLM rewriter (HyDE-style), and A/B it against the hand lexicon on the paraphrase split.
5. **Guardrail depth**: add a small injection classifier and evaluate on paraphrased attacks; measure PII redaction precision/recall on a labelled sample.
6. **Scale test**: generate a 10^4-document synthetic corpus and profile retrieval, then introduce an inverted index and re-profile.

10. Learning Guide

10.1 Concept checklist — you understand this project if you can explain

[ ] TF vs IDF, and why IDF uses a logarithm and smoothing

[ ] Why L2 normalisation turns a dot product into cosine similarity

[ ] BM25's two constants (k1 saturation, b length normalisation) with the formula

[ ] Why min-max normalising BM25 would break abstention, and what a saturating transform does instead

[ ] First-stage vs second-stage retrieval; bi-encoder vs cross-encoder

[ ] Query expansion: benefits, drift risk, and auditable vs learned variants

[ ] Extractive vs abstractive composition; faithfulness vs correctness

[ ] How a token-overlap verifier works and what it cannot catch

[ ] Absolute-score abstention and threshold calibration

[ ] Recall@k, and why a paraphrase split discriminates better than a main split

[ ] Ablation design: change one factor, keep everything else identical

[ ] Ceiling effects in tuning grids and why tie-breaks then matter

[ ] Prompt injection families and the limits of pattern defence

[ ] Luhn validation and why redaction rule order matters

[ ] Agent decomposition: contracts, traces, and where vetoes belong

10.2 Interview Q&A (model answers)

Q1. Why not just fine-tune an LLM on the company policies instead of building RAG?
Fine-tuning bakes knowledge into weights: it is slow and costly to repeat on every policy change, it cannot cite its source, and it still hallucinates on out-of-scope questions. RAG keeps knowledge in an index — update the documents and the answers update — and gives citations and an inspectable retrieval step. Fine-tuning is better for *behaviour and style*; RAG is better for *facts*. Production systems often use both.

Q2. Explain BM25 in one minute. BM25 scores a document for a query by summing, over query terms, an IDF weight times a saturated, length-normalised term frequency: f·(k1+1) / (f + k1·(1 b + b·|D|/avgdl)). k1 (1.5) makes repeated mentions count with diminishing returns; b (0.75) stops long documents winning just by containing more words. Its IDF, ln(1 + (N DF + 0.5)/(DF + 0.5)), rewards rare terms. It remains the strongest classical lexical ranker, which is why hybrid stacks keep it next to embeddings.

Q3. Your TF-IDF baseline got 100% on the main set. Why build anything more? Because the main set asks questions in the documents' own vocabulary, so it cannot discriminate — every method saturates. The paraphrase split asks the same facts in user vocabulary ("fake email", "login secret"); there TF-IDF drops to 85.71% Recall@3, hybrid + expansion reaches 92.86%, and reranking reaches 100%. The lesson generalises: if your eval can't make your baseline fail, your eval is wrong, not your baseline perfect.

Q4. How does hybrid fusion work here, and what's the normalisation trap? Fused = 0.55·cosine + 0.45·(BM25/(BM25+8)). The trap is min-max normalising BM25 per query: it forces the top score to

~1.0 even for nonsense queries, destroying the absolute scale the abstention threshold depends on. The saturating transform keeps 0 at 0, so "nothing matched" stays numerically visible.

Q5. What is reranking and why not rerank the whole corpus? A reranker scores query–chunk pairs *jointly* (cross-encoder in production; here a deterministic cross-scorer using coverage, best-sentence overlap and phrase bonus). Joint scoring is more accurate but expensive, so it runs only on the top-20 pool from the cheap first stage. In the ablation its effect is exactly the textbook one: it doesn't change what's *found* (R@5 already 100% after hybrid) — it fixes *order* at the top (R@3 92.86% 100%).

Q6. How do you stop the system hallucinating? Three layers. The composer is extractive — answers are verbatim retrieved sentences with citations, so it cannot invent facts. The Verifier agent independently checks every answer sentence for 50% content-token overlap with the cited chunks, and the Orchestrator replaces a failing answer with an abstention. And the threshold makes the system abstain when retrieval evidence is weak in the first place. If the composer is swapped for an LLM, layers two and three remain — that's the point of the design.

Q7. Groundedness was 100% — isn't that circular for extractive answers? Partly, and the report says so: extractive sentences are grounded by construction. The metric's role is regression detection — it proves the invariant holds through the whole orchestrated path today, and it becomes the critical safety metric the day an abstractive generator replaces the composer. Grounding also isn't correctness: a faithfully quoted *wrong* chunk passes verification, which is why retrieval recall is measured separately.

Q8. How was the abstention threshold chosen, and what's its weakness? By sweeping candidate thresholds to maximise joint accuracy: answerable questions (42) should score τ, unanswerable (5) below τ. $\tau = 0.20$ achieved 47/47 — but the separating band is narrow (answerable min 0.2105, unanswerable max 0.1919), so borderline future questions can flip. Production fix: more calibration data and a learned calibrator, or a two-band answer-with-caveat zone.

Q9. How does prompt-injection defence work, and where does it fail? Fifteen regex families cover instruction override, persona hijack, developer-mode and prompt-exfiltration phrasings; a match blocks before any agent runs, and blocking is measured end-to-end (5/5). It fails on novel paraphrases of the same ideas — patterns catch known phrasings. Hence "defence-in-depth": in production you'd add a classifier model, least-privilege tool design, and human review, and never let retrieved text become instructions.

Q10. Why redact PII on the *output* as well as the input? Inputs can contain PII that would otherwise be echoed, logged in traces, or — with an LLM composer — repeated into an answer. Redacting the final answer closes the loop regardless of which component produced the text. The redactor uses ordered rules and a Luhn check for cards so ordinary long numbers (order IDs, policy numbers) aren't destroyed, and treats 4-4-4 grouped 12-digit numbers as Aadhaar-shaped rather than cards.

Q11. How would you scale this to a million documents? Replace the two brute-force scorers behind their existing contracts: embeddings go into an ANN index (FAISS/pgvector), BM25 into an inverted index; fusion and reranking are unchanged (reranking is already pool-based, which is the scalable pattern). Chunking becomes structure-aware, the threshold becomes a trained calibrator, and the evaluation harness becomes a regression suite run on every index/config change.

Q12. What did the tuning grid actually teach you, given five configurations tied at 100%? That the benchmark had saturated: once Recall hits ceiling, the grid can't rank configurations, so the

tie-break — fewest chunks, i.e. the simplest index — makes the decision, and the honest conclusion is "the evaluation now needs harder questions", not "200/20 is uniquely best". Recognising a ceiling effect is a core experimental skill.

Q13. Why implement BM25 and TF-IDF from scratch instead of using a library? Two reasons: zero-dependency reproducibility (the whole system runs anywhere with Python + NumPy, and every number in the report can be regenerated), and understanding — implementing the IDF variants, saturation and normalisation choices forces the design decisions (like the saturating transform) into the open, where they can be explained and defended in review or interview. The interfaces keep library/neural replacements one class away.

10.3 Exercises to extend the project

1. **Held-out split**: write 14 new questions (half paraphrased) without looking at the lexicon; measure all three ablation modes. How far does the full stack fall from 100%?
2. **Stemming**: add a light suffix stemmer to `content_tokens` and re-run the ablation. Which paraphrase questions does it fix that expansion missed — and does it break any main-set question?
3. **Tune α**: sweep fusion α ∈ {0.3, 0.45, 0.55, 0.7} and the BM25 saturating constant ∈ {4, 8, 16}; plot paraphrase R@3 and the threshold separation band. Does the band widen?
4. **Break the verifier**: hand-write an abstractive answer containing one invented number; confirm the verifier fails it. Then write one that paraphrases so heavily it fails despite being correct — what does that teach about token-overlap verification?
5. **Novel injection**: craft three injection prompts that evade all 15 patterns; then extend the patterns (or add a heuristic) to catch them without blocking any of the 42 legitimate questions.
6. **Cross-document question**: add one question needing facts from two documents (e.g. password rules *and* lockout-driven helpdesk steps). What breaks — retrieval, composition, or verification — and what's the minimal fix?
7. **Scale profile**: duplicate the corpus with renamed synthetic variants to ~1,400 documents; measure latency; then add a simple inverted index for BM25 and re-measure.

---

11. How to Run

Requirements: Python 3.12, NumPy. No API keys, no downloads.

```
cd advanced-ai-projects/agentforge-rag

python3 tests.py        # 25 assert-based tests — expect "All 25 tests passed."
python3 demo.py         # 4 traced end-to-end queries (policy, incident, paraphrase, abstain)
python3 evaluate.py     # tuning sweep + ablation -> results.json (regenerates all report
numbers)
python3 api.py 8000     # HTTP API on 127.0.0.1:8000

# API smoke test:
curl http://127.0.0.1:8000/health
curl -X POST http://127.0.0.1:8000/ask \
     -H 'Content-Type: application/json' \
     -d '{"question": "What is the 3-2-1 backup rule?", "top_k": 5}'
```

Reading order for learners: `guardrails.py` (self-contained) `rag.py` (§4.1 order) `agents.py` `evaluate.py` `tests.py`.

12. References

1. Lewis, P., Perez, E., Piktus, A., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.* NeurIPS 2020.

2. Robertson, S., & Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond.* Foundations and Trends in Information Retrieval, 3(4).

3. Robertson, S., & Walker, S. (1994). *Some Simple Effective Approximations to the 2-Poisson Model for Probabilistic Weighted Retrieval (BM25).* SIGIR 1994.

4. Jones, K. S. (1972). *A Statistical Interpretation of Term Specificity and Its Application in Retrieval.* Journal of Documentation, 28(1).

5. Salton, G., Wong, A., & Yang, C. S. (1975). *A Vector Space Model for Automatic Indexing.* Communications of the ACM, 18(11).

6. Karpukhin, V., Oguz, B., Min, S., et al. (2020). *Dense Passage Retrieval for Open-Domain Question Answering.* EMNLP 2020.

7. Nogueira, R., & Cho, K. (2019). *Passage Re-ranking with BERT.* arXiv:1901.08149.

8. Gao, L., Ma, X., Lin, J., & Callan, J. (2022). *Precise Zero-Shot Dense Retrieval without Relevance Labels (HyDE).* arXiv:2212.10496.

9. Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2023). *Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection.* arXiv:2310.11511.

10. Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2023). *RAGAS: Automated Evaluation of Retrieval Augmented Generation.* arXiv:2309.15217.

11. Rebedea, T., Dinu, R., Sreedhar, M., Parisien, C., & Cohen, J. (2023). *NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails.* EMNLP 2023 (demo).

12. OWASP Foundation. *OWASP Top 10 for LLM Applications* (LLM01: Prompt Injection). owasp.org.

13. Luhn, H. P. (1960). *Computer for Verifying Numbers.* US Patent 2,950,048.

Appendix A — Full `results.json`

Verbatim output of `python3 evaluate.py` for the final run (generated, never hand-edited):


```
{
  "config": {
    "chunk_words": 200,
    "overlap_words": 20,
    "abstention_threshold": 0.2,
    "n_chunks": 28
  },
  "tuning": {
    "chosen": {
      "chunk_words": 200,
      "overlap_words": 20,
      "abstention_threshold": 0.2,
      "n_chunks": 28
    },
    "threshold_accuracy_on_eval": 1.0,
    "min_answerable_top_score": 0.2105,
    "max_unanswerable_top_score": 0.1919,
    "grid_top5": [
      {
        "chunk_words": 200,
        "overlap_words": 20,
        "n_chunks": 28,
        "recall_at_5_combined": 1.0,
        "recall_at_3_combined": 1.0
      },
      {
        "chunk_words": 200,
        "overlap_words": 30,
        "n_chunks": 28,
        "recall_at_5_combined": 1.0,
        "recall_at_3_combined": 1.0
      },
      {
        "chunk_words": 200,
        "overlap_words": 40,
        "n_chunks": 28,
        "recall_at_5_combined": 1.0,
        "recall_at_3_combined": 1.0
      },
      {
        "chunk_words": 160,
        "overlap_words": 20,
        "n_chunks": 42,
        "recall_at_5_combined": 1.0,
        "recall_at_3_combined": 1.0
      },
      {
        "chunk_words": 160,
        "overlap_words": 30,
        "n_chunks": 42,
        "recall_at_5_combined": 1.0,
        "recall_at_3_combined": 1.0
      }
    ]
  },
  "corpus": {
    "n_documents": 14,
    "n_chunks": 28
  },
  "eval_set_size": 28,
  "paraphrase_set_size": 14,
  "ablation": {
    "naive_tfidf_only": {
      "main_recall_at_3": 1.0,
      "main_recall_at_5": 1.0,
      "paraphrase_recall_at_3": 0.8571,
      "paraphrase_recall_at_5": 0.9286
```

```
    },
    "hybrid_tfidf_bm25_expansion": {
      "main_recall_at_3": 1.0,
      "main_recall_at_5": 1.0,
      "paraphrase_recall_at_3": 0.9286,
      "paraphrase_recall_at_5": 1.0
    },
    "hybrid_plus_rerank": {
      "main_recall_at_3": 1.0,
      "main_recall_at_5": 1.0,
      "paraphrase_recall_at_3": 1.0,
      "paraphrase_recall_at_5": 1.0
    }
  },
  "metrics": {
    "recall_at_3": 1.0,
    "recall_at_5": 1.0,
    "paraphrase_recall_at_3": 1.0,
    "paraphrase_recall_at_5": 1.0,
    "groundedness_rate": 1.0,
    "answered_questions": 28,
    "abstention_correctness": 1.0,
    "unanswerable_questions": 5,
    "injection_block_rate": 1.0,
    "injection_block_rate_end_to_end": 1.0,
    "attack_prompts": 5,
    "mean_latency_ms": 10.56,
    "median_latency_ms": 5.29
  },
  "per_question": [
    {
      "question": "How many characters must a password be at least and how often do passwords
expire?",
      "expected_doc": "password-policy",
      "hit_at_3": true,
      "hit_at_5": true,
      "abstained": false
    },
    {
      "question": "What happens after 5 consecutive failed login attempts and how long is the
account locked?",
      "expected_doc": "password-policy",
      "hit_at_3": true,
      "hit_at_5": true,
      "abstained": false
    },
    {
      "question": "Does remote access over the VPN require multi factor authentication and when
does a session disconnect for inactivity?",
      "expected_doc": "vpn-remote-access-policy",
      "hit_at_3": true,
      "hit_at_5": true,
      "abstained": false
    },
    {
      "question": "What are the device requirements for a laptop to connect over the VPN and
what happens if the health check fails?",
      "expected_doc": "vpn-remote-access-policy",
      "hit_at_3": true,
      "hit_at_5": true,
      "abstained": false
    },
    {
      "question": "What is a Severity 1 incident and within how many minutes must the response
start?",
      "expected_doc": "incident-response-playbook",
      "hit_at_3": true,
      "hit_at_5": true,
      "abstained": false
    },
    {
      "question": "Within how many hours must the regulator be notified of a confirmed personal
data breach?",
```

```
      "expected_doc": "incident-response-playbook",
      "hit_at_3": true,
      "hit_at_5": true,
      "abstained": false
    },
    {
      "question": "How should an employee report a phishing message and what button should they
use?",
      "expected_doc": "phishing-response-playbook",
      "hit_at_3": true,
      "hit_at_5": true,
      "abstained": false
    },
    {
      "question": "What happens if an employee entered credentials on a fake phishing page?",
      "expected_doc": "phishing-response-playbook",
      "hit_at_3": true,
      "hit_at_5": true,
      "abstained": false
    },
    {
      "question": "What is the company policy on paying a ransom in a ransomware attack?",
      "expected_doc": "ransomware-response-playbook",
      "hit_at_3": true,
      "hit_at_5": true,
      "abstained": false
    },
    {
      "question": "In what priority order are systems recovered from backups after ransomware,
starting with domain controllers?",
      "expected_doc": "ransomware-response-playbook",
      "hit_at_3": true,
      "hit_at_5": true,
      "abstained": false
    },
    {
      "question": "What are the four data classification levels and what is Restricted data?",
      "expected_doc": "data-classification-and-handling-policy",
      "hit_at_3": true,
      "hit_at_5": true,
      "abstained": false
    },
    {
      "question": "What does the data loss prevention system monitor email and web uploads
for?",
      "expected_doc": "data-classification-and-handling-policy",
      "hit_at_3": true,
      "hit_at_5": true,
      "abstained": false
    },
    {
      "question": "Is cryptocurrency mining on company hardware a prohibited use?",
      "expected_doc": "acceptable-use-policy",
      "hit_at_3": true,
      "hit_at_5": true,
      "abstained": false
    },
    {
      "question": "How can employees install new software and where must software be installed
from?",
      "expected_doc": "acceptable-use-policy",
      "hit_at_3": true,
      "hit_at_5": true,
      "abstained": false
    },
    {
      "question": "How is administrator access granted under the principle of least privilege
and what is a separate privileged account?",
      "expected_doc": "access-control-and-privilege-policy",
      "hit_at_3": true,
      "hit_at_5": true,
      "abstained": false
    },
```

```
    {
      "question": "How often are access reviews done for production systems and when are
inactive accounts disabled?",
      "expected_doc": "access-control-and-privilege-policy",
      "hit_at_3": true,
      "hit_at_5": true,
      "abstained": false
    },
    {
      "question": "What is the 3-2-1 backup rule for copies of data and media types?",
      "expected_doc": "backup-and-disaster-recovery-policy",
      "hit_at_3": true,
      "hit_at_5": true,
      "abstained": false
    },
    {
      "question": "What is the Recovery Time Objective RTO for critical customer systems?",
      "expected_doc": "backup-and-disaster-recovery-policy",
      "hit_at_3": true,
      "hit_at_5": true,
      "abstained": false
    },
    {
      "question": "Within how many hours must a lost or stolen BYOD device be reported to the
helpdesk?",
      "expected_doc": "bring-your-own-device-policy",
      "hit_at_3": true,
      "hit_at_5": true,
      "abstained": false
    },
    {
      "question": "What can the company wipe from a personal BYOD device and what is the work
container?",
      "expected_doc": "bring-your-own-device-policy",
      "hit_at_3": true,
      "hit_at_5": true,
      "abstained": false
    },
    {
      "question": "Within how many days must new employees complete security awareness
training?",
      "expected_doc": "security-awareness-and-training-policy",
      "hit_at_3": true,
      "hit_at_5": true,
      "abstained": false
    },
    {
      "question": "How often are simulated phishing campaigns run and what happens after a
click?",
      "expected_doc": "security-awareness-and-training-policy",
      "hit_at_3": true,
      "hit_at_5": true,
      "abstained": false
    },
    {
      "question": "Within how many days must a critical vulnerability with CVSS 9.0 on an
internet facing system be patched?",
      "expected_doc": "vulnerability-management-policy",
      "hit_at_3": true,
      "hit_at_5": true,
      "abstained": false
    },
    {
      "question": "How long do medium severity vulnerabilities have to be patched?",
      "expected_doc": "vulnerability-management-policy",
      "hit_at_3": true,
      "hit_at_5": true,
      "abstained": false
    },
    {
      "question": "How many business days before a start date must Human Resources notify IT
about a new joiner onboarding?",
      "expected_doc": "employee-onboarding-and-offboarding-policy",
```

```
      "hit_at_3": true,
      "hit_at_5": true,
      "abstained": false
    },
    {
      "question": "Within how many business days must equipment be returned during offboarding
after the last day?",
      "expected_doc": "employee-onboarding-and-offboarding-policy",
      "hit_at_3": true,
      "hit_at_5": true,
      "abstained": false
    },
    {
      "question": "How long are customer account records retained after an account closes?",
      "expected_doc": "data-retention-and-privacy-policy",
      "hit_at_3": true,
      "hit_at_5": true,
      "abstained": false
    },
    {
      "question": "Within how many days must a privacy rights request for access to personal
data be answered?",
      "expected_doc": "data-retention-and-privacy-policy",
      "hit_at_3": true,
      "hit_at_5": true,
      "abstained": false
    }
  ]
}
```

Appendix B — File Listing

Project root: `advanced-ai-projects/agentforge-rag/`

- `README.md` (9451 bytes)
- `agents.py` (5864 bytes)
- `api.py` (2777 bytes)
- `corpus/`
- `corpus/acceptable-use-policy.md` (2186 bytes)
- `corpus/access-control-and-privilege-policy.md` (2146 bytes)
- `corpus/backup-and-disaster-recovery-policy.md` (1995 bytes)
- `corpus/bring-your-own-device-policy.md` (2232 bytes)
- `corpus/data-classification-and-handling-policy.md` (2262 bytes)
- `corpus/data-retention-and-privacy-policy.md` (2167 bytes)
- `corpus/employee-onboarding-and-offboarding-policy.md` (2057 bytes)
- `corpus/incident-response-playbook.md` (2326 bytes)
- `corpus/password-policy.md` (2217 bytes)
- `corpus/phishing-response-playbook.md` (2165 bytes)
- `corpus/ransomware-response-playbook.md` (2169 bytes)
- `corpus/security-awareness-and-training-policy.md` (2408 bytes)
- `corpus/vpn-remote-access-policy.md` (2372 bytes)
- `corpus/vulnerability-management-policy.md` (2426 bytes)
- `demo.py` (1757 bytes)
- `evaluate.py` (13612 bytes)
- `gen_corpus.py` (31033 bytes)
- `guardrails.py` (4824 bytes)
- `rag.py` (16643 bytes)
- `results.json` (9377 bytes)
- `tests.py` (4244 bytes)

End of report — AgentForge-RAG, Ashutosh Sahoo, NIT Silchar, November 2026.